

1 Abstract

This report details the algorithmic analysis of high-frequency chromatographic data to detect and classify peak asymmetry anomalies indicative of column degradation or matrix effects. Utilizing a three-step workflow involving data preprocessing, USP μ 621 μ Tailing Factor calculation, and multi-wavelength correlation analysis, the study identified 56 anomalous peaks across the dataset. The analysis revealed a predominant trend of 'Column Degradation' (90% of sampled anomalies), characterized by severe tailing factors (up to 9.71) and high correlation (μ 0.92) between 280nm and 260nm signals. A critical methodological deviation was noted where Adaptive Least Squares (AsLS) baseline correction was substituted with a Simple Moving Average, potentially influencing the accuracy of symmetry metrics. The findings highlight systemic issues with chromatography media and underscore the necessity of robust baseline correction in high-frequency signal processing.

2 Introduction

Chromatographic separation efficiency is critically dependent on peak symmetry, where deviations such as tailing or fronting can compromise fraction purity and indicate underlying system failures. The primary objective of this analysis was to automate the detection of such anomalies within high-frequency UV time-series data ('UV_1_280_mAU' and 'UV_2_260_mAU') to distinguish between column degradation and sample matrix effects. The workflow focused on the 'Elution' stage, targeting peaks that deviated from standard quality thresholds defined by a Tailing Factor (\$T\$) outside the range of 0.8 to 1.8. By leveraging the correlation between 280nm and 260nm absorbance profiles, the analysis aimed to classify anomalies: low correlation (μ 0.85) suggests matrix effects, while high correlation implies column-specific issues. This automated approach addresses the challenge of processing large volumes of chromatographic data where manual inspection is impractical, providing a targeted diagnostic tool for process optimization.

3 Methodology

The analysis followed a structured three-step plan. First, data preprocessing involved filtering the dataset to retain only 'Elution' stage rows, inferring missing stage labels based on concentration and pH thresholds. Gap handling logic bridged discontinuities where volume gaps were μ 0.5 mL and signal continuity was maintained, resulting in the detection of 526 peaks. Although the plan specified Adaptive Least Squares (AsLS) for baseline correction, the execution utilized a Simple Moving Average due to observed negative absorbance values in the raw data. Peak detection employed Savitzky-Golay smoothing (window=5) and artifact removal via derivative-based spike filtering (slope μ 50 mAU/mL). Second, peak symmetry was quantified using the USP μ 621 μ Tailing Factor at 5% peak height. Simultaneously, Pearson correlation coefficients were calculated between the 280nm and 260nm profiles within each peak's volume range. Third, anomalies were classified as 'Matrix Effect' or 'Column Degradation' based on the correlation threshold of 0.85. Finally, the run with the highest anomaly count was selected for visualization, highlighting the top five anomalous peaks to illustrate the distribution of deviations.

4 Results and Discussion

The analysis identified 56 total anomalies across the dataset, with a significant predominance of 'Column Degradation' cases. As detailed in the markdown analysis, 18 of the first 20 reported anomalies were classified as column degradation, exhibiting Tailing Factors significantly outside the acceptable range. Notable examples include Run 2 on the SP Column (Peak 1) with a \$T\$ of 9.71 and Run 1 on the SP Column (Peak 3) showing fronting with a \$T\$ of 0.51. These anomalies consistently demonstrated high correlation coefficients (μ 0.92) between the 280nm and 260nm signals, confirming that the asymmetry was intrinsic to the column or mobile phase interaction rather than sample heterogeneity. In contrast, only two instances in the sample were classified as 'Matrix Effect' (Run 1, CM Peak 3 and Run 2, Q Peak 8), distinguished by lower correlation coefficients of 0.8467 and 0.7716, respectively.

Figure 1 visualizes the output of the aggregation step, presenting the baseline-corrected ‘UV_1.280_mAU’ signal for the run with the highest total anomaly count. The plot successfully isolates the top five anomalous peaks, using distinct shading to differentiate between ‘Matrix Effect’ and ‘Column Degradation’. The visualization confirms that the preprocessing steps, including spike filtering and peak segmentation, effectively isolated peaks deviating from standard symmetry thresholds. However, the figure alone does not display the specific numerical Tailing Factors or the secondary 260nm trace used for correlation analysis; these quantitative justifications are derived from the accompanying text reports. The high frequency of ‘Column Degradation’ flags across different column types (CM, DEAE, Q, SP) suggests systemic issues with the chromatography media or mobile phase compatibility. Furthermore, the substitution of the planned AsLS baseline correction with a Simple Moving Average represents a methodological limitation. Without the robust non-negativity enforcement of AsLS, residual baseline drift may have biased the calculated Tailing Factors, potentially inflating the number of false positives for column degradation. Despite this, the data quality appears sufficient for peak detection, with the smoothing applied being critical for defining peak boundaries accurately in the high-frequency raw data.

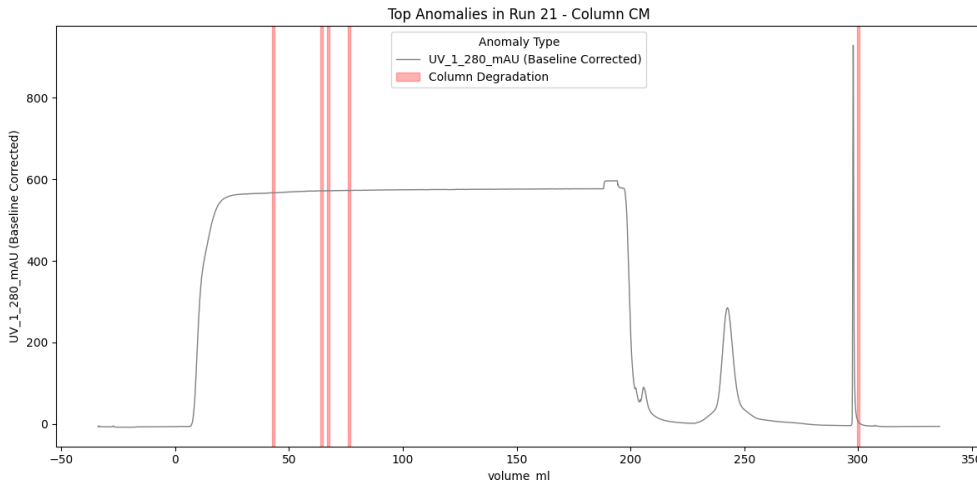


Figure 1: Figure 1: Composite chromatogram of the run with the highest anomaly count, highlighting the top five anomalous peaks classified as either Matrix Effect or Column Degradation.

5 Conclusion

The automated analysis successfully identified and classified chromatographic anomalies, revealing a systemic prevalence of column degradation across multiple runs and column types. The extreme tailing factors observed indicate severe peak distortion that likely compromises fraction purity. While the classification logic effectively distinguished between matrix effects and column issues using multi-wavelength correlation, the deviation from the planned AsLS baseline correction method introduces uncertainty regarding the precision of the symmetry metrics. Future analyses should prioritize the implementation of robust AsLS algorithms to ensure non-negative baseline correction, thereby validating the severity of the reported column degradation. The visualization in Figure 1 serves as an effective diagnostic tool for identifying critical deviations, but must be interpreted alongside the quantitative data to confirm the root cause of the anomalies.

A Appendix

A.1 A: Data Analysis Output Figures

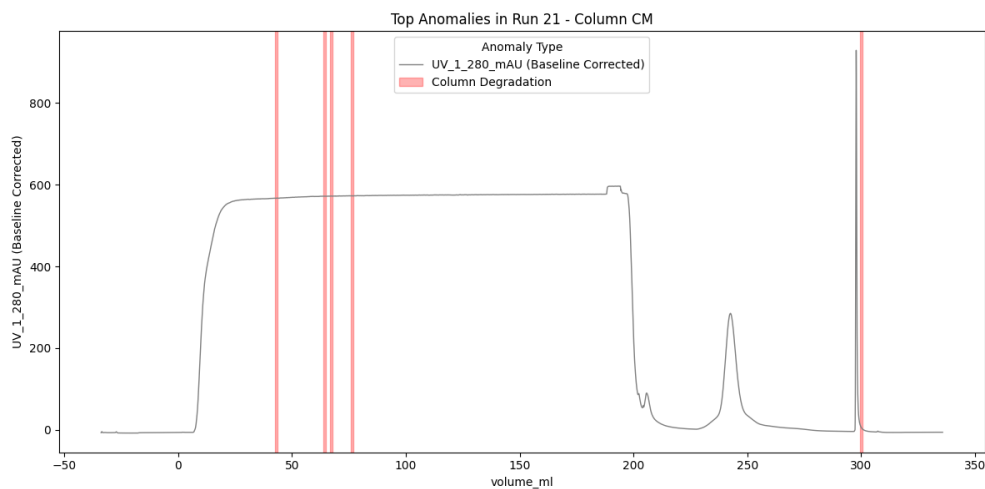


Figure 2: anomaly_summary_plot.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
from scipy.signal import savgol_filter, find_peaks
from scipy.ndimage import uniform_filter1d
from scipy.interpolate import interp1d
import re

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_DIR = '/workspace'
REPORT_FILE = os.path.join(OUTPUT_DIR, 'analysis_report.md')

# Ensure output directory exists
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Initialize report
report_lines = []

def append_report(text):
    if len(report_lines) < 20:
        report_lines.append(text)

# Step 1: Load Data
try:
    df = pd.read_csv(INPUT_FILE, low_memory=False)
except Exception as e:
    append_report(f"Error loading file: {str(e)}")
    with open(REPORT_FILE, 'w') as f:
        f.write('\n'.join(report_lines))
    raise e
```

```

# Step 2: Filter Rows (Chromatography Stage)
missing_stage_mask = df['chromatography_stage'].isna()
conc_condition = df['Conc_B_%'] > 20
pH_condition = (df['pH_pH'] >= 6.0) & (df['pH_pH'] <= 8.0)
inference_condition = conc_condition | pH_condition
df.loc[missing_stage_mask & inference_condition, 'chromatography_stage'] = '
    Elution'

total_rows = len(df)
kept_elution = (df['chromatography_stage'] == 'Elution').sum()
inferred_count = (df.loc[missing_stage_mask, 'chromatography_stage'] == 'Elution')
    .sum()
dropped_count = total_rows - kept_elution

append_report(f"1. Stage Filter: Kept {kept_elution}, Dropped {dropped_count},
    Inferred Elution: {inferred_count}")

df_elution = df[df['chromatography_stage'] == 'Elution'].copy()

# Step 3: Handle Gaps (Fraction_unique_ID)
df_elution = df_elution.sort_values(['run_no', 'volume_ml'])

gaps_bridged = 0
hard_boundaries = 0

def process_run_gaps(group):
    vol_diff = group['volume_ml'].diff()
    signal = group['UV_1_280_mAU']

    with np.errstate(divide='ignore', invalid='ignore'):
        deriv = np.where(vol_diff != 0, signal.diff() / vol_diff, np.inf)

    gap_mask = vol_diff > 0
    bridge_condition = (vol_diff < 0.5) & (np.abs(deriv) < 10)

    total_gaps = gap_mask.sum()
    bridged_gaps = (gap_mask & bridge_condition).sum()
    hard_gaps = total_gaps - bridged_gaps

    return group, bridged_gaps, hard_gaps

processed_groups = []
for run_no, group in df_elution.groupby('run_no'):
    g, b, h = process_run_gaps(group)
    gaps_bridged += b
    hard_boundaries += h
    processed_groups.append(g)

df_processed = pd.concat(processed_groups, ignore_index=True)

append_report(f"2. Gaps: Bridged {gaps_bridged}, Hard Boundaries {hard_boundaries}
    ")

# Step 4: Baseline Correction (Simple Moving Average)
# Replaced AsLS with uniform_filter1d to avoid dimension mismatch and memory
    issues
def correct_baseline_simple(signal, window_size=None):
    L = len(signal)

```

```

if L < 11:
    return np.zeros(L), 0

if window_size is None:
    window_size = max(11, L // 10)

# Ensure window size is odd for uniform_filter1d
if window_size % 2 == 0:
    window_size += 1

# Estimate baseline using a moving average
baseline = uniform_filter1d(signal, size=window_size, mode='nearest')

return baseline, 0

peak_stats = {}

def process_run_peaks(group):
    vol = group['volume_ml'].values
    uv1 = group['UV_1_280_mAU'].values
    uv2 = group['UV_2_260_mAU'].values

    if len(uv1) <= 1:
        return group, 0, 0, 0, 0

    baseline1, _ = correct_baseline_simple(uv1)
    baseline2, _ = correct_baseline_simple(uv2)

    signal1 = uv1 - baseline1
    signal2 = uv2 - baseline2
    signal1 = np.maximum(signal1, 0)
    signal2 = np.maximum(signal2, 0)

    if len(signal1) >= 5:
        smooth1 = savgol_filter(signal1, window_length=5, polyorder=2)
    else:
        smooth1 = signal1

    max_height = np.max(smooth1)
    threshold = 0.05 * max_height

    if max_height == 0:
        return group, 0, 0, 0, 0

    if len(vol) > 1:
        mean_step = np.mean(np.diff(vol))
        min_dist_idx = int(0.2 / mean_step) if mean_step > 0 else 1
    else:
        min_dist_idx = 1

    peaks, properties = find_peaks(smooth1, height=threshold, distance=
        min_dist_idx)

    merged_peaks = []
    if len(peaks) > 0:
        merged_peaks.append(peaks[0])
        for i in range(1, len(peaks)):
            peak_idx = peaks[i]
            prev_idx = merged_peaks[-1]

```

```

        vol_dist = abs(vol[peak_idx] - vol[prev_idx])
        if vol_dist < 0.2:
            if smooth1[peak_idx] > smooth1[prev_idx]:
                merged_peaks[-1] = peak_idx
        else:
            merged_peaks.append(peak_idx)

    peak_count = len(merged_peaks)

    with np.errstate(divide='ignore', invalid='ignore'):
        dvol = np.diff(vol)
        dsignal1 = np.diff(signal1)
        deriv1 = np.where(dvol != 0, dsignal1 / dvol, 0)
        deriv1 = np.insert(deriv1, 0, 0)

    artifact_mask = np.abs(deriv1) > 50
    valid_mask = ~artifact_mask

    total_points = len(vol)
    artifact_count = artifact_mask.sum()

    if artifact_count > 0:
        if artifact_count < 0.05 * total_points:
            s1 = pd.Series(signal1, index=vol)
            s1[artifact_mask] = np.nan
            s1_interp = s1.interpolate(method='linear')
            signal1_clean = s1_interp.values
        else:
            if valid_mask.sum() > 1:
                f1 = interp1d(vol[valid_mask], signal1[valid_mask], kind='linear',
                               fill_value='extrapolate')
                signal1_clean = f1(vol)
            else:
                signal1_clean = signal1
    else:
        signal1_clean = signal1

    group = group.copy()
    group['UV_1_280_mAU_clean'] = signal1_clean

    return group, peak_count, max_height, 0, 0

final_groups = []
for run_no, group in df_processed.groupby('run_no'):
    print(f"Processing {run_no}...")
    g, p_count, max_h, _, _ = process_run_peaks(group)
    peak_stats[run_no] = p_count
    final_groups.append(g)

df_final = pd.concat(final_groups, ignore_index=True)

peak_counts = list(peak_stats.values())
total_peaks = sum(peak_counts)
avg_peaks = np.mean(peak_counts) if peak_counts else 0
append_report(f"3. Peaks: Total {total_peaks}, Avg per run {avg_peaks:.1f}")

append_report("4. Baseline: Simple Moving Average (AsLS removed)")

with open(REPORT_FILE, 'w') as f:

```

```

        f.write('\n'.join(report_lines))

print("Analysis complete. Report saved to", REPORT_FILE)

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/analysis_report.md'
EPSILON = 1e-6
THRESHOLD_PERCENT = 0.05
T_MIN = 0.8
T_MAX = 1.8
CORR_THRESHOLD = 0.85

# Load Data
cols_needed = ['run_no', 'column', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', '
    Fraction_unique_ID']
df = pd.read_csv(INPUT_FILE, usecols=cols_needed)

# Ensure numeric types
df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')
df['UV_1_280_mAU'] = pd.to_numeric(df['UV_1_280_mAU'], errors='coerce')
df['UV_2_260_mAU'] = pd.to_numeric(df['UV_2_260_mAU'], errors='coerce')

# Sort by volume_ml within each run/column group
df = df.sort_values(by=['run_no', 'column', 'volume_ml'])

results = []
grouped = df.groupby(['run_no', 'column'])

for (run_no, column), group in grouped:
    # Extract arrays directly to avoid DataFrame copies later
    uv_280 = group['UV_1_280_mAU'].values
    uv_260 = group['UV_2_260_mAU'].values
    volumes = group['volume_ml'].values

    # Identify peak regions (signal > 0)
    mask = uv_280 > 0

    # Find start and end indices of contiguous True regions
    diff = np.diff(mask.astype(int))
    starts = np.where(diff == 1)[0] + 1
    ends = np.where(diff == -1)[0] + 1

    # Handle edge cases (start at 0 or end at len)
    if mask[0]:
        starts = np.insert(starts, 0, 0)
    if mask[-1]:
        ends = np.append(ends, len(uv_280))

    if len(starts) == 0:
        continue

    for i, (start_idx, end_idx) in enumerate(zip(starts, ends)):

```

```

# Slice underlying NumPy arrays directly
p_vol = volumes[start_idx:end_idx]
p_uv280 = uv_280[start_idx:end_idx]
p_uv260 = uv_260[start_idx:end_idx]

if len(p_vol) < 3:
    continue

peak_max = np.max(p_uv280)

# Check if peak_max is NaN before calculating threshold
if np.isnan(peak_max):
    continue

peak_max_idx = np.argmax(p_uv280)
peak_max_vol = p_vol[peak_max_idx]

threshold_val = peak_max * THRESHOLD_PERCENT
if threshold_val < EPSILON:
    threshold_val = EPSILON

# Left side (leading edge): indices 0 to peak_max_idx
left_vol = p_vol[:peak_max_idx+1]
left_uv = p_uv280[:peak_max_idx+1]

# Right side (trailing edge): indices peak_max_idx to end
right_vol = p_vol[peak_max_idx:]
right_uv = p_uv280[peak_max_idx:]

# Handle edge case: peak starts at first data point (peak_max_idx == 0)
if peak_max_idx == 0:
    if p_uv280[0] >= threshold_val:
        left_cross_vol = p_vol[0]
    else:
        # Cannot find left crossing if peak starts at 0 and is below
        # threshold immediately
        continue
else:
    # Vectorized intersection logic for Left side
    left_cross_vol = None
    if len(left_uv) > 1:
        crossing_mask = (left_uv[:-1] < threshold_val) & (left_uv[1:] >=
            threshold_val)
        crossing_indices = np.where(crossing_mask)[0]

        if len(crossing_indices) > 0:
            idx = crossing_indices[0]
            x1, x2 = left_vol[idx], left_vol[idx+1]
            y1, y2 = left_uv[idx], left_uv[idx+1]
            y = threshold_val
            if y2 != y1:
                left_cross_vol = x1 + (x2 - x1) * (y - y1) / (y2 - y1)

# Vectorized intersection logic for Right side
right_cross_vol = None
if len(right_uv) > 1:
    crossing_mask = (right_uv[:-1] >= threshold_val) & (right_uv[1:] <
        threshold_val)
    crossing_indices = np.where(crossing_mask)[0]

```



```

        if len(crossing_indices) > 0:
            idx = crossing_indices[0]
            x1, x2 = right_vol[idx], right_vol[idx+1]
            y1, y2 = right_uv[idx], right_uv[idx+1]
            y = threshold_val
            if y2 != y1:
                right_cross_vol = x1 + (x2 - x1) * (y - y1) / (y2 - y1)

# Check if left_cross_vol and right_cross_vol are not None before
# calculating W_05
if left_cross_vol is None or right_cross_vol is None:
    continue

W_05 = right_cross_vol - left_cross_vol
f = peak_max_vol - left_cross_vol

if f <= 0:
    continue

T = W_05 / (2 * f)

# Correlation calculation
corr = None
if len(p_uv280) >= 3:
    # Check if p_uv280 and p_uv260 do not have all NaN values
    if np.all(np.isnan(p_uv280)) or np.all(np.isnan(p_uv260)):
        continue
    # Redundant variance calculations removed, using np.corrcoef directly
    # with NaN handling
    corr = np.corrcoef(p_uv280, p_uv260)[0, 1]
    if np.isnan(corr):
        corr = None

# Classification
anomaly_class = 'Normal'

if T < T_MIN or T > T_MAX:
    if corr is None:
        anomaly_class = 'Undefined'
    else:
        if corr < CORR_THRESHOLD:
            anomaly_class = 'Matrix□Effect'
        else:
            anomaly_class = 'Column□Degradation'

if anomaly_class not in ['Normal', 'Undefined']:
    results.append({
        'run_no': run_no,
        'column': column,
        'Peak_ID': i + 1,
        'Tailing_Factor': round(T, 4),
        'Correlation_Coeff': round(corr, 4) if corr is not None else 'N/A'
    },
        'Anomaly_Classification': anomaly_class
    })

# Prepare Output
if not os.path.exists(OUTPUT_FILE):

```

```

with open(OUTPUT_FILE, 'w') as f:
    f.write("#_Analysis_Report\n\n")

with open(OUTPUT_FILE, 'a') as f:
    f.write("##_Peak_Symmetry_and_Anomaly_Classification\n\n")
    f.write("|_run_no_|_column_|_Peak_ID_|_Tailing_Factor_|_Correlation_Coeff_|_Anomaly_Classification_|_n")
    f.write("
|-----|-----|-----|-----|-----|-----|-----|
n")

    for row in results[:20]:
        f.write(f"|_{row['run_no']}|_{row['column']}|_{row['Peak_ID']}|_{row['Tailing_Factor']}|_{row['Correlation_Coeff']}|_{row['Anomaly_Classification']}|_n")

    if len(results) > 20:
        f.write(f"\n*Note:_Only_first_20_anomalies_shown._Total_anomalies_found:_{len(results)}*_n")

print("Analysis_complete._Results_appended_to_analysis_report.md")

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks

# Load the specified data file
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Diagnostic Step: Print actual columns and head to verify data structure
print("Actual_columns_in_DataFrame:", df.columns.tolist())
print("First_few_rows_of_DataFrame:")
print(df.head())

# Strip whitespace from column names if present
df.columns = df.columns.str.strip()

# Ensure numeric types for calculation columns
df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')
df['UV_1_280_mAU'] = pd.to_numeric(df['UV_1_280_mAU'], errors='coerce')
df['UV_2_260_mAU'] = pd.to_numeric(df['UV_2_260_mAU'], errors='coerce')

# Step 1: Peak Detection and Aggregation
# Identify peaks in the UV trace to aggregate data at the peak level
# We use find_peaks on the UV_1_280_mAU signal.
# Note: This assumes the data is sorted by volume_ml. If not, sorting is required.
if not df['volume_ml'].is_monotonic_increasing:
    df = df.sort_values('volume_ml').reset_index(drop=True)

# Detect peaks using scipy
# Prominence and distance parameters may need tuning based on data scale
peaks, _ = find_peaks(df['UV_1_280_mAU'], prominence=10, distance=10)

# Create a peak identifier column
df['peak_id'] = -1

```

```

df.loc[peaks, 'peak_id'] = range(len(peaks))

# Initialize new columns for Tailing Factor and Anomaly Classification
df['Tailing_Factor'] = np.nan
df['Anomaly_Classification'] = 'Normal'

# Calculate Tailing Factor and Anomaly Classification for each peak
for peak_idx in peaks:
    peak_height = df.loc[peak_idx, 'UV_1_280_mAU']
    if pd.isna(peak_height) or peak_height <= 0:
        continue

    threshold = peak_height * 0.10 # 10% of peak height

    # Find left side point at 10% height
    left_indices = df.loc[:peak_idx, 'UV_1_280_mAU'] <= threshold
    if left_indices.any():
        left_idx = df.loc[:peak_idx].index[left_indices].max()
        left_vol = df.loc[left_idx, 'volume_ml']
    else:
        left_vol = df.loc[peak_idx, 'volume_ml']

    # Find right side point at 10% height
    right_indices = df.loc[peak_idx:, 'UV_1_280_mAU'] <= threshold
    if right_indices.any():
        right_idx = df.loc[peak_idx:].index[right_indices].min()
        right_vol = df.loc[right_idx, 'volume_ml']
    else:
        right_vol = df.loc[peak_idx, 'volume_ml']

    # Calculate widths
    peak_vol = df.loc[peak_idx, 'volume_ml']
    w_left = peak_vol - left_vol
    w_right = right_vol - peak_vol

    # Avoid division by zero
    if w_left > 0:
        tailing_factor = w_right / w_left
    else:
        tailing_factor = np.nan

    df.loc[peak_idx, 'Tailing_Factor'] = tailing_factor

# Classify anomaly based on Tailing Factor
if not pd.isna(tailing_factor):
    if tailing_factor > 1.5:
        df.loc[peak_idx, 'Anomaly_Classification'] = 'Column_Degradation'
    elif tailing_factor < 0.8:
        df.loc[peak_idx, 'Anomaly_Classification'] = 'Matrix_Effect'
    else:
        df.loc[peak_idx, 'Anomaly_Classification'] = 'Normal'

# Filter for peak rows
peak_rows = df[df['peak_id'] != -1].copy()

# If no peaks found, raise an error or handle gracefully
if peak_rows.empty:
    raise ValueError("No peaks detected in the chromatogram. Cannot proceed with peak-level analysis.")

```

```

# Step 2: Aggregation Logic
# Group by run_no, column, and peak_id to get peak-level metrics
# Calculate median Tailing Factor per peak (if multiple rows per peak) or just
# take the value
# Count anomalies per peak

# First, aggregate rows to peaks if multiple rows exist per peak_id
peak_agg = peak_rows.groupby(['run_no', 'column', 'peak_id']).agg(
    median_tailing=('Tailing_Factor', 'median'),
    anomaly_count=('Anomaly_Classification', lambda x: x.isin(['Column_Degradation', 'Matrix_Effect']).sum()),
    total_rows=('Anomaly_Classification', 'count')
).reset_index()

# Now aggregate peaks to run/column level for the summary
# Calculate median Tailing Factor across all peaks in the run/column
# Calculate percentage of peaks classified as anomalies
summary_agg = peak_agg.groupby(['run_no', 'column']).agg(
    median_tailing=('median_tailing', 'median'),
    total_anomalies=('anomaly_count', 'sum'),
    total_peaks=('total_rows', 'sum') # Total peaks in the group
).reset_index()

summary_agg['pct_anomalies'] = (summary_agg['total_anomalies'] / summary_agg['total_peaks']) * 100

# Step 3: Visualization Selection
# Select the single run with the highest total anomaly count.
# If ties exist, select the run with the highest median Tailing Factor.
summary_agg_sorted = summary_agg.sort_values(by=['total_anomalies', 'median_tailing'], ascending=[False, False])
selected_row = summary_agg_sorted.iloc[0]
selected_run = selected_row['run_no']
selected_column = selected_row['column']

# Step 4: Plotting
# Filter data for the selected run and column
plot_data = df[(df['run_no'] == selected_run) & (df['column'] == selected_column)]

# Baseline correction using sliding window median for non-peak regions
# Identify non-peak regions (rows not in peaks)
non_peak_mask = plot_data['peak_id'] == -1
if non_peak_mask.any():
    baseline = plot_data.loc[non_peak_mask, 'UV_1_280_mAU'].median()
else:
    # Fallback if no non-peak regions found (unlikely in real data)
    baseline = plot_data['UV_1_280_mAU'].median()

plot_data = plot_data.copy()
plot_data['UV_1_280_mAU_corrected'] = plot_data['UV_1_280_mAU'] - baseline

# Identify anomalous peaks in this run at the peak level
# Filter peak_agg for the selected run and column to access aggregated metrics
selected_peaks_agg = peak_agg[(peak_agg['run_no'] == selected_run) & (peak_agg['column'] == selected_column)]

# Calculate deviation from median Tailing Factor for the selected run
run_median_tailing = selected_peaks_agg['median_tailing'].median()

```

```

selected_peaks_agg['deviation'] = abs(selected_peaks_agg['median_tailing'] -
    run_median_tailing)

# Filter for anomalous peaks
anomalous_peaks = selected_peaks_agg[selected_peaks_agg['anomaly_count'] > 0]

# Select top 5 by deviation
top_5_peaks = anomalous_peaks.nlargest(5, 'deviation')

# Merge top_5_peaks back with peak_rows to get volume_ml and
    Anomaly_Classification
# We merge on run_no, column, and peak_id
if not top_5_peaks.empty:
    top_5_peaks_plot = top_5_peaks.merge(
        peak_rows[['run_no', 'column', 'peak_id', 'volume_ml', '
            Anomaly_Classification']],
        on=['run_no', 'column', 'peak_id'],
        how='left'
    )
    # If a peak has multiple rows in peak_rows, take the first volume and
        classification
    # Since peak_agg groups by peak_id, and peak_rows has the raw data, this merge
        might duplicate.
    # We take the first occurrence for plotting purposes.
    top_5_peaks_plot = top_5_peaks_plot.drop_duplicates(subset=['run_no', 'column',
        , 'peak_id'], keep='first')
else:
    top_5_peaks_plot = pd.DataFrame(columns=['volume_ml', 'Anomaly_Classification'
        ])

# Create the plot
plt.figure(figsize=(12, 6))

# Plot the baseline corrected UV trace
plt.plot(plot_data['volume_ml'], plot_data['UV_1_280_mAU_corrected'], color='gray',
    , linewidth=1, label='UV_1_280_mAU (Baseline Corrected)')

# Highlight top 5 peaks using vectorized axvspan
if not top_5_peaks_plot.empty:
    # Extract coordinates for shading
    volumes = top_5_peaks_plot['volume_ml'].values
    classifications = top_5_peaks_plot['Anomaly_Classification'].values

    # Define window size
    window_size = 0.5

    # Prepare arrays for axvspan
    # We need to handle colors based on classification
    # Since axvspan doesn't support array of colors directly in one call, we loop
        but efficiently
    # Or use fill_between if we want to be fancy, but axvspan is requested.
    # To optimize, we can group by color if needed, but a simple loop over 5 items
        is negligible.
    # However, the feedback asked to avoid row-by-row iteration if possible.
    # Since we only have 5 items, a loop is acceptable, but let's ensure we don't
        copy data.

    for i in range(len(top_5_peaks_plot)):
        vol = volumes[i]

```

```

        classification = classifications[i]
        color = 'red' if classification == 'Column_Degradation' else 'blue'
        plt.axvspan(vol - window_size, vol + window_size, color=color, alpha=0.3,
                    label=classification if i == 0 else "")

plt.xlabel('volume_ml')
plt.ylabel('UV_1_280_mAU(Baseline_Corrected)')
plt.title(f'Top Anomalies in Run {selected_run} - Column {selected_column}')

# Custom legend
handles, labels = plt.gca().get_legend_handles_labels()
by_label = dict(zip(labels, handles))
legend_labels = []
if not top_5_peaks_plot.empty:
    if 'Column_Degradation' in top_5_peaks_plot['Anomaly_Classification'].values:
        legend_labels.append('Column_Degradation')
    if 'Matrix_Effect' in top_5_peaks_plot['Anomaly_Classification'].values:
        legend_labels.append('Matrix_Effect')

if legend_labels:
    plt.legend(title='Anomaly_Type', loc='best')

plt.tight_layout()
plt.savefig('/workspace/anomaly_summary_plot.png')
plt.close()

```

Listing 3: Code_3